

4. Relay R2 구성 정리

4. Relay R2

4-1. 이 프로세스의 정체

Relay R2는 R1에서 받은 이벤트를 후단으로 전달하는 두 번째 relay 노드다.

주요 역할은:

- R1이 forward한 EVENT를 수신
- 중복 여부 검사
- 필요하면 후단 노드(Monitor)로 전달
- ACK 반환
- 설정에 따라 failover/alternate path 지원
- 후단 전달 실패 시 retry 또는 보류

즉 R2는 후단 relay다.

4-2. 왜 필요한가

R1 하나만 있어도 전달은 된다.

하지만 그러면 실험 범위가 약하다.

R2가 있으면 아래를 보여줄 수 있다.

- relay hop 증가
- 중간 노드 간 ACK/retry
- relay-to-relay dedup
- 후단 전달 실패 처리
- relay 장애 시 대체 전달 구조
- 1차 relay와 2차 relay의 역할 분리

즉 분산 전달 네트워크라는 느낌이 살아난다.

4-3. R2의 목적

목적은 5개다.

1. R1이 보낸 이벤트 수신

- relay chain의 두 번째 hop

2. 중복 검사

- 재전송 때문에 같은 event_id가 다시 올 수 있음

3. Monitor로 전달

- 최종 sink에 넘김

4. R1에 ACK 반환

- "R2가 정상적으로 받았다"를 알려줌

5. 후단 전달 정책 관리

- Monitor 지연/장애 시 retry 또는 buffering
 - failover/direct path 같은 후속 정책 적용 가능
-

4-4. 입력 / 출력

입력

A. R1에서 오는 EVENT

예:

```
{
  "event_id": "evt-host1-000041",
  "seq_no": 41,
  "host_id": "host-1",
  "agent_id": "agent-host-1",
  "type": "CPU_SPIKE",
  "severity": "WARN",
  "timestamp": "2026-03-19T10:00:13",
  "payload": {
    "cpu": 95
  }
}
```

B. Monitor에서 오는 ACK

설계에 따라 Monitor가 ACK를 줄 수 있다.

R2가 후단 전달 보장까지 말한다면 이 ACK가 중요하다.

예:

```
{
  "msg_type": "ACK",
  "ack_for": "evt-host1-000041",
  "from": "monitor"
}
```

C. Controller/UI 제어 명령

예:

- pause
- down
- add delay
- drop forward
- failover enabled
- direct monitor path enabled

D. 내부 타이머

- 후단 ACK timeout 검사
 - queue 처리
 - cache 정리
-

출력

A. R1으로 보내는 ACK

- R2 수신 성공 알림

B. Monitor로 보내는 EVENT

- 최종 sink로 전달

C. 로그/통계

- duplicate
- forwarded
- timeout
- retry
- final failure
- buffered

D. 상태 보고

- pending ack 수
 - queue 길이
 - 후단 전달 성공률
 - fallback 사용 횟수
-

4-5. R2 내부 상태값

R1과 거의 유사하지만, 후단 전달과 failover용 상태가 조금 더 중요하다.

```
node_id
running_state
```

```
downstream_target
ack_timeout_ms
max_retry_count
incoming_queue
outgoing_queue
pending_ack_table
received_id_cache
retry_counter_table
processing_delay_ms
drop_mode
failover_enabled
alternate_target
buffer_on_failure
```

각 의미를 정리한다.

node_id

- `r2`

running_state

- `RUNNING`, `PAUSED`, `DOWN`

downstream_target

- 보통 Monitor 주소

ack_timeout_ms

- Monitor ACK 대기 시간

max_retry_count

- 후단 전송 재시도 한도

incoming_queue

- R1에서 받은 메시지 처리 대기

outgoing_queue

- Monitor로 보낼 메시지 대기

pending_ack_table

- Monitor ACK 대기 중인 메시지

received_id_cache

- dedup용 ID cache

retry_counter_table

- 후단 전송 재시도 횟수

processing_delay_ms

- 실험용 지연

drop_mode

- 일부 메시지/ACK 드롭 모드

failover_enabled

- 후단 장애 시 대체 정책 사용 여부

alternate_target

- direct monitor 외 다른 sink나 대체 경로가 있을 때 사용

buffer_on_failure

- 후단 전송 실패 시 메시지 보관 여부

4-6. R2가 담당하는 핵심 기능

기능 1. R1 메시지 수신

목적

R1의 forward 결과를 받아 relay chain을 이어감

작동 방식

- 메시지 수신
- 형식 검증
- dedup 검사 전 단계 진입

실제 작동

```
R1 → R2
R2: received evt-host1-000041
```

기능 2. 유효성 검증

목적

메시지가 깨졌거나 필수 필드가 없으면 후단 전달 방지

검사 항목

- event_id

- seq_no
- host_id
- type
- timestamp

실제 작동

문제가 있으면:

- 로그 기록
- drop
- R1에는 에러 또는 무응답 정책 가능

초기 구현은 단순 drop + log면 충분하다.

기능 3. dedup 검사

목적

R1의 retry로 인해 동일 메시지가 여러 번 들어오는 상황 처리

작동 방식

- `received_id_cache`에 event_id가 있으면 duplicate
- 후단 재전달은 생략
- R1에는 ACK만 다시 반환 가능

실제 작동

```
evt-host1-000041
이미 처리됨
→ duplicate 판단
→ Monitor로 재전달 안 함
→ R1에 ACK 재전송 가능
```

기능 4. R1에 ACK 반환

목적

R1이 timeout/retry 판단을 할 수 있도록 수신 성공 알림

작동 방식

R2가 메시지를 정상 수신하고 accepted 하면 ACK 전송

예:

```
{
  "msg_type": "ACK",
  "ack_for": "evt-host1-000041",
```

```
"from": "r2"
}
```

의미

이 ACK는 "R2 수신 성공"이다.

Monitor 최종 기록 완료와는 별개일 수 있다.

이 부분은 설계 선택이다.

기능 5. Monitor로 forward

목적

Relay chain의 마지막 단계에서 최종 sink로 전달

작동 방식

중복 아니고 유효한 메시지를 Monitor로 전송

실제 작동

```
R2 accepted evt-host1-000041
→ forward to Monitor
```

기능 6. 후단 ACK 대기

목적

Monitor까지의 최종 전달을 추적

작동 방식

Monitor가 ACK를 주는 설계라면:

- 전송 직후 `pending_ack_table` 등록
- timeout 검사
- retry 수행

이 설계를 쓰면 R2는 **후단 전달 보장 노드**가 된다.

초기 구현에서는 이 구조가 더 명확하다.

기능 7. 후단 retry

목적

Monitor 지연/장애 시 메시지 손실 감소

작동 방식

- ACK timeout 발생

- retry_count 증가
- Monitor에 재전송

실제 작동

```
R2 → Monitor
ACK 없음
→ timeout
→ resend
```

기능 8. 최종 실패 처리 또는 buffer

목적

후단 장애 시 메시지 처리 정책 결정

선택 가능한 정책

A. 실패 처리

- max retry 초과 시 drop + failed log

B. buffer

- max retry 초과 후 보류 큐에 저장
- Monitor 복구 시 재전달

초기 구현은 A가 단순하고,
확장 버전은 B가 더 실험적으로 좋다.

기능 9. failover / alternate path 지원

목적

Monitor 또는 후단 경로 장애 시 대체 정책을 적용

가능한 형태

- direct path
- backup sink
- local buffer
- alternate monitor node

실제 작동 예

```
Monitor ACK 없음
retry 초과
```



```
failover_enabled=true  
→ alternate_target으로 전달 시도
```

현재 프로젝트에서 꼭 복잡하게 할 필요는 없고,

“후단 실패 시 **alternate target** 또는 **buffer**로 전환할 수 있다” 정도면 충분하다.

기능 10. 실험용 장애 주입 수용

목적

Controller에서 R2를 일부러 느리게/불안정하게 만들기 위함

지원하면 좋은 옵션

- processing delay
- forward drop
- ack drop
- pause
- down
- queue overflow mode

실제 효과

- R1 timeout 증가
- retry 증가
- duplicate 증가
- end-to-end delay 증가

즉 실험 포인트가 된다.

4-7. R2의 실제 동작 흐름

정상 전달 흐름

1. R1이 EVENT 전송
2. R2 수신
3. 유효성 검사
4. dedup 검사
5. duplicate 아니면 accepted
6. R1에 ACK 반환
7. Monitor로 forward
8. Monitor ACK 수신
9. 전송 완료 처리

duplicate 수신 흐름

1. R1이 동일 event_id 재전송
2. R2 수신
3. received_id_cache 확인
4. 이미 처리한 메시지임
5. Monitor 재전달 생략
6. R1에 ACK만 반환

Monitor ACK 유실 흐름

1. R2가 Monitor로 메시지 전송
2. Monitor는 실제 수신 및 기록
3. ACK가 유실됨
4. R2 timeout 발생
5. 동일 메시지 재전송
6. Monitor는 event_id 기준으로 duplicate 판단
7. 실제 저장 생략
8. ACK만 재전송
9. R2 완료 처리

이 시나리오는 Monitor 쪽 dedup도 필요하게 만든다.

Monitor down 흐름

1. R2가 Monitor로 전송
2. 응답 없음
3. timeout
4. retry 반복
5. max retry 초과
6. 정책 판단:
 - 실패 처리
 - buffer 저장
 - alternate target 사용

4-8. R1과 R2의 차이

기능은 비슷하지만, 의미상 차이가 있다.

R1의 성격

- Agent 바로 뒤
- 1차 relay
- 로컬 이벤트를 네트워크 전달로 넘기는 첫 노드

R2의 성격

- 2차 relay
- 최종 sink 직전
- 후단 전달 보장
- failover 설명의 중심 노드

즉 R2는 후단 전송 안정성 쪽이 더 중요하다.

4-9. R2가 가져야 하는 로그/통계

기본 통계

- total_received_from_r1
- total_forwarded_to_monitor
- duplicate_dropped
- downstream_timeout_count
- downstream_retry_total
- downstream_failed_count
- buffered_count
- fallback_used_count

이 통계가 있어야 다음 비교가 가능하다.

- R2 지연 시 retry 증가
 - Monitor down 시 failed_count 증가
 - buffer_on_failure on/off 비교
 - failover 사용 횟수 확인
-

4-10. R2의 정책 파라미터

Controller/UI에서 바꿀 수 있으면 좋다.

필수

- ack_timeout_ms
- max_retry_count
- dedup_enabled
- processing_delay_ms

있으면 좋은 것

- buffer_on_failure

- failover_enabled
- alternate_target
- queue_limit
- monitor_ack_required

예:

- `monitor_ack_required=false` 로 두면 단순 fire-and-forget처럼 동작
- `true` 면 실제 후단 전달 보장 성격이 강해짐

4-11. R2가 하지 않는 것

R2도 모든 걸 하진 않는다.

하지 않는 것:

- Host 상태 감지 안 함
- 이벤트 생성 안 함
- 사용자 UI 렌더링 안 함
- 최종 로그 분석 UI 직접 제공 안 함

즉 R2는 **후단 relay**지, 최종 분석 화면 그 자체는 아니다.

4-12. R2만 따로 구조도로 그리면

Relay R2
입력 - EVENT from R1 - ACK from Monitor - Control from Controller/UI
내부 상태 - incoming_queue - outgoing_queue - pending_ack_table - received_id_cache - retry_counter - buffer_on_failure - alternate_target
내부 기능 - receive - validate - dedup check - send ACK to R1

- forward to Monitor
- wait downstream ACK
- timeout / retry
- buffer or failover

출력

- ACK to R1
- EVENT to Monitor
- log/stat updates

4-13. 앞단과 연결해서 보면

```
Host Simulator
  ↓
Local Agent
  ↓
Relay R1
  ↓
Relay R2
  ↓
Monitor
```

여기서 R2는:

- relay chain의 마지막 relay
- 후단 전달 보장 및 장애 대응 노드

라고 보면 된다.

4-14. 핵심 한 문장 정리

Relay R2는 Relay R1이 전달한 이벤트를 수신하여 중복을 검사하고, Monitor로 전달한 뒤 후단 ACK를 관리하며, 필요 시 retry, buffering, failover 같은 후단 전달 정책을 수행하는 2차 relay 프로세스다.

4-15. 지금까지 네 프로세스 흐름 요약

```
Host Simulator
- 상태/장애 생성

↓ 상태 조회

Local Agent
- polling
- 이벤트 생성
```

- R1 publish

↓ EVENT

Relay R1

- 수신
- dedup
- R2 forward
- ACK wait / retry

↓ EVENT

Relay R2

- 수신
- dedup
- Monitor forward
- downstream ACK wait / retry / buffer / failover

다음이라